

Day 1: Edge AI landscape and system constraints

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 1, 2026

1. Block 1: Why inference moves to the edge
2. Block 2: Deep learning recap from a cost view
3. Block 3: The edge AI workflow

- ▶ A model that runs in the cloud does not always run on a small device. Memory, power, and latency set hard limits.
- ▶ You must know the limits of a device before you select a model for it.
- ▶ In the lab you set up the XIAOML Kit and test its sensors. Then you calculate if three models fit four devices.
- ▶ This day gives the terms and the budgets that all later days use.

After this day you can:

- ▶ Explain the four deployment paradigms and the constraints that separate them.
- ▶ Estimate the parameters, the operations, and the peak activation memory of a model.
- ▶ Decide from flash and RAM numbers if a model fits a device.
- ▶ Set up the toolchain and test each sensor of the XIAOML Kit.

Week	Subject	Board	Days
1	Foundations and TinyML	XIAOML Kit	Landscape, sensor data, model deployment, quantization, audio and vision
2	Optimization and Linux-class edge devices	Laptop, Raspberry Pi 5	Pruning and distillation, runtimes, object detection, benchmarking, generative AI
3	Connected systems and production	Both boards	RTSP streaming, MQTT, monitoring, production design, capstone

Key point: You optimize a model, deploy it on two device classes, connect the devices, and monitor them. The capstone project combines all parts into one system.

Theory: three blocks

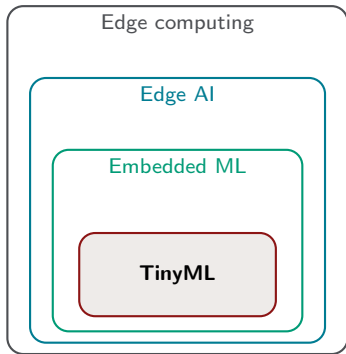
- ▶ Block 1: the main concept of the day.
- ▶ Block 2: methods and tools.
- ▶ Block 3: design trade-offs and the lab briefing.
- ▶ Each block has one short exercise.

Lab: groups of two students

- ▶ You work on the hardware in two to four parts.
- ▶ You predict a number before you measure it.
- ▶ At the end you show the result to the instructor.
- ▶ You write a Decision Log.

Decision Log

A text of about 100 words. You state one design decision, you give the measured numbers, and you name the trade-off.



Edge computing Computation on a device at the edge of the network, near the data source.

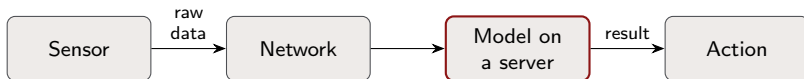
Edge AI A model makes the decision on the edge device.

Embedded ML Inference on an embedded system. Training stays on a large computer.

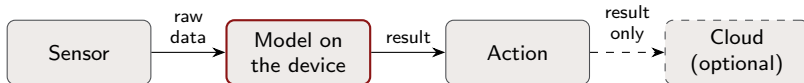
TinyML Inference on the smallest hardware: a microcontroller with kilobytes of memory.

Two paths for inference

Cloud path



Edge path



- ▶ Training needs much computation. It stays on large computers.
- ▶ Inference can move to the device. Then the device needs no permanent connection.
- ▶ The cloud path has three problems: raw data uses bandwidth, each transmission uses power, and the sensor must stay connected.

Five reasons to move inference to the edge

Reason	The question that you ask
Latency	How much time can pass between the input and the response?
Bandwidth	How much data does the sensor make each second? Can the connection carry it?
Privacy	Is the data permitted to leave the device?
Cost	What does each inference cost in the cloud? What does each device cost?
Availability	Must the system work when the network fails?

Key point: One of these constraints usually decides. Find that constraint first.

Source: The five factors of Jeff Bier ("BLERP": bandwidth, latency, economics, reliability, privacy), from *XIAO: Big Power, Small Board*, chapter 4.1.

Latency: the speed of light sets a minimum

$$t_{\min} = \frac{2 \times \text{distance}}{c_{\text{fiber}}}$$

California to Virginia: 3600 km.

$$t_{\min} \approx \frac{2 \times 3600 \text{ km}}{200\,000 \text{ km/s}} = 36 \text{ ms}$$

- ▶ Software adds 60 to 150 ms.
- ▶ A cloud service needs 100 to 200 ms in total.
- ▶ A faster network does not change the 36 ms.

Application	Budget
Vehicle braking	< 10 ms
Game rendering	< 16 ms
Robot control	< 20 ms
Voice response	< 200 ms
Web search	< 500 ms

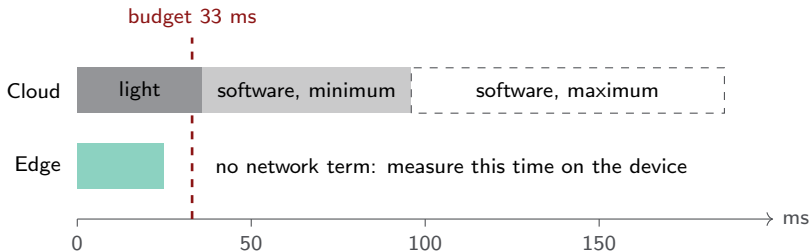
Source: *Machine Learning Systems*, Volume I, chapter 2.

Latency: use a budget for the complete path

Rule

If the budget is X ms, no operation that needs more than X ms can be on the path from the input to the response.

Example: a camera gives 30 frames each second. The system must answer before the next frame: $1000/30 \approx 33$ ms.



Source: Rule and cloud numbers: *Machine Learning Systems*, Volume I, chapter 2. The length of the edge bar is an example, not a measured value.

A car with 8 cameras

- ▶ 30 frames each second
 - ▶ 2 MB for each frame
- $$8 \times 30 \times 2 \text{ MB} = 480 \text{ MB/s}$$

No mobile connection carries this data rate.

A microphone on a microcontroller

- ▶ 16 000 samples each second
 - ▶ 2 bytes for each sample
- $$16\,000 \times 2 \text{ B} = 32 \text{ kB/s}$$

A keyword result is one short message.

- ▶ A site with hundreds of cameras cannot send all video to the cloud.
- ▶ Each transmitted byte also uses energy. A radio is expensive for a battery device.

- ▶ Cameras and microphones record persons.
- ▶ Data in the cloud passes through networks and servers.
- ▶ Some data must stay on the site: patient data, images of a production line.
- ▶ An edge device sends only the result, or nothing.

Paradigm	Privacy	The data
Cloud	Low	leaves the device
Edge	High	stays on the site
Mobile	High	stays on the phone
TinyML	Highest	stays in the sensor

Key point: More shared computation gives less privacy. Data that stays local cannot use the computation of a data centre.

Cost

- ▶ Cloud computation and communication cost money for each inference. Video is the most expensive.
- ▶ An edge device sends less data and uses less cloud computation.
- ▶ A microcontroller costs 1 to 10 USD for each unit.
- ▶ The edge has its own costs: hardware maintenance and device management.

Availability

- ▶ A cloud service stops when the connection stops.
- ▶ A device with a local model continues to work.
- ▶ Example: a door lock with face recognition must open for the family when the internet fails.
- ▶ On Day 12 you build a system that operates offline.

Source: Unit cost: *Machine Learning Systems*, Volume I, chapter 2.

Why not one paradigm for all applications?

Light barrier

Light in fiber travels about 200 000 km/s.

California to Virginia and back: 36 ms minimum.

A response in 10 ms cannot use a distant server.

Power wall

The power grows with the third power of the clock frequency.

Two times the frequency needs eight times the power.

A phone has 3 to 5 W. It cannot run a data-centre model.

Memory wall

Computation grows 1.6 times each year. Memory bandwidth grows 1.2 times.

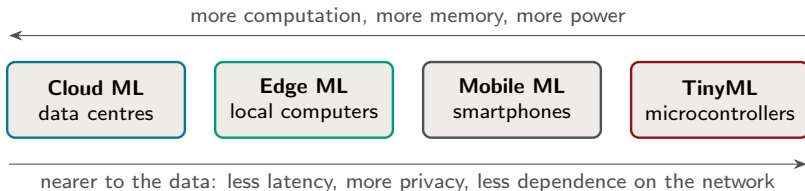
To move data often costs more than to compute on it.

A microcontroller has 256 KB of RAM.

Key point: These limits are physics. No optimization removes them. They make four paradigms.

Source: *Machine Learning Systems*, Volume I, chapter 2.

Four deployment paradigms



Paradigm	Memory	Power	Latency	Privacy	Network
Cloud	80 GB	700 W	100 to 1000 ms	Low	Necessary
Edge	32 GB	15 to 40 W	10 to 100 ms	High	Intermittent
Mobile	8 GB	3 to 5 W	5 to 50 ms	High	Optional
TinyML	256 KB	100 mW	1 to 10 ms	Highest	None

Source: Table: *Machine Learning Systems*, Volume I, chapter 2. Each row gives one typical device of the class.

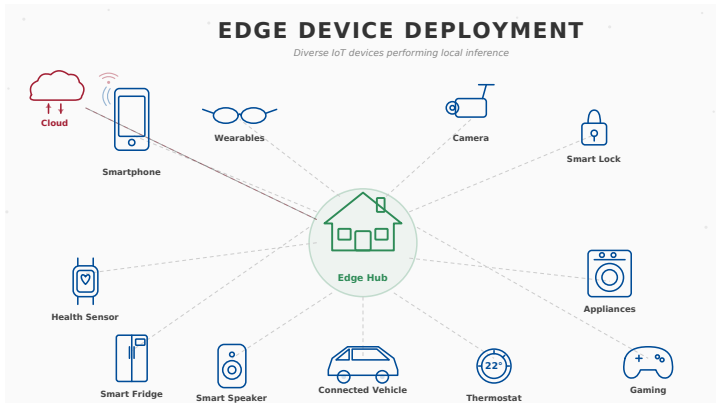
Strengths

- ▶ Computation and storage grow with the demand.
- ▶ Very large data sets.
- ▶ Large models, for example large language models.
- ▶ Training of almost all models.

Limits

- ▶ Latency: each request crosses the network.
- ▶ The data leaves the device.
- ▶ The system needs a connection.
- ▶ The cost increases with each request.

Key point: Use the cloud when the application accepts the latency, the data can leave the device, and the model needs the computation.



- ▶ A local computer or a gateway runs the model. The data stays on the site.
- ▶ Limit: less computation than the cloud. **In this course:** the Raspberry Pi 5.

Source: Figure: *Machine Learning Systems*, Volume I, chapter 2 (CC BY-NC-SA 4.0).

- ▶ The model runs on a smartphone or a tablet.
- ▶ Advantages: it works offline, the data stays with the user, and the latency is low.
- ▶ The power limit is 3 to 5 W. The device has no fan.
- ▶ Above this limit the processor decreases its clock speed.
Example: 60 frames each second decrease to 15 in one minute.

Battery calculation

A battery of 15 Wh:

- ▶ at 5 W: $15/5 = 3$ hours
- ▶ at 1 W: $15/1 = 15$ hours

Key point: Mobile ML needs models that are designed for this limit, for example MobileNetV2. Block 2 shows why such a model costs less.

Source: *Machine Learning Systems*, Volume I, chapter 2.

TinyML: a different class, not a small phone

	Mobile	TinyML
Memory	8 GB	256 KB
Power	3 to 5 W	100 mW
Parameters	millions	10 000 to 100 000

Memory: a factor of about 30 000.

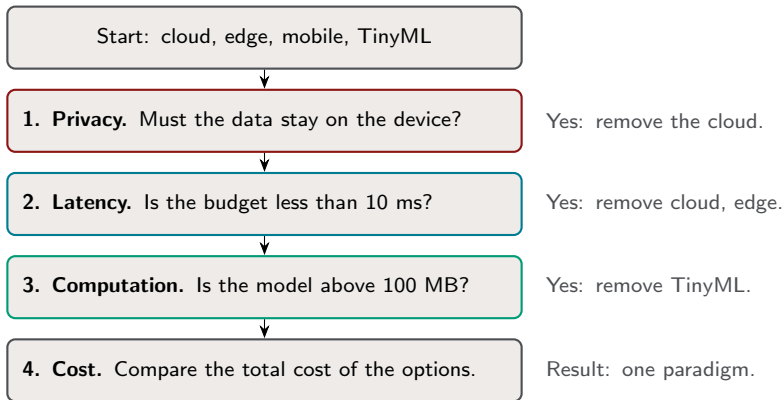
Power: a factor of 30 to 50.

- ▶ The model runs on a microcontroller, next to the sensor.
- ▶ The device is always on. It can use a battery.
- ▶ The energy of each inference and the memory decide, not the speed.
- ▶ Example: a keyword spotting model has 26 000 parameters and less than 100 KB.

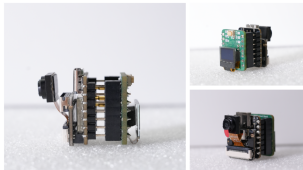
Key point: You cannot make a mobile model smaller until it fits. TinyML needs different models and different tools.

Source: *Machine Learning Systems*, Volume I, chapter 2.

How to select a paradigm

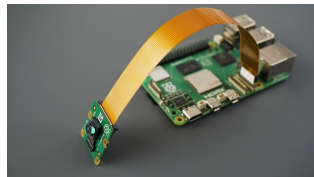


Apply the filters in order. Each filter removes the paradigms that are not possible. The method does not start with the “best” paradigm.



XIAOML Kit (TinyML)

- ▶ A microcontroller board with Wi-Fi and Bluetooth
- ▶ Camera, microphone, microSD card
- ▶ Expansion board: IMU and display
- ▶ Board size: 21×17.5 mm



Raspberry Pi 5 (Edge ML)

- ▶ A single-board computer with 8 GB of RAM
- ▶ Linux, Python, complete ML frameworks
- ▶ Camera connector, USB, Ethernet, Wi-Fi
- ▶ In the course: with a camera and an active cooler

Source: Photographs and data: kit pages of *Machine Learning Systems* (CC BY-NC-SA 4.0).

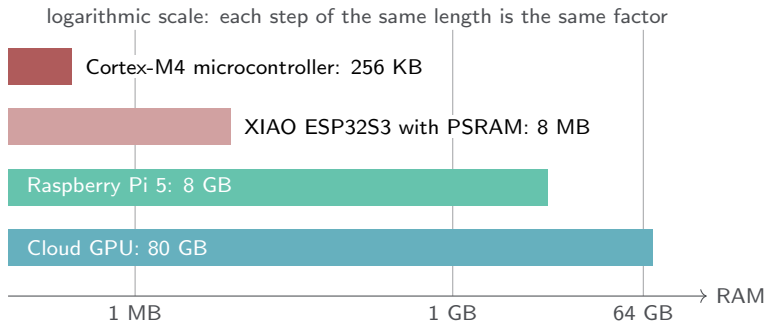
Four devices, four budgets

	Cortex-M4 microcontroller	XIAO ESP32S3 Sense	Raspberry Pi 5	Cloud GPU
Processor	Cortex-M4F, 1 core, 64 MHz	Xtensa LX7, 2 cores, 240 MHz	Cortex-A76, 4 cores, 2.4 GHz	GPU in a server
RAM	256 KB	512 KB, and 8 MB of PSRAM	8 GB	80 GB on the GPU
Storage	1 MB of flash	8 MB of flash	microSD card	Almost no limit
Software	Firmware in C++	Firmware in C++, or MicroPython	Linux, Python	Linux, Python
Paradigm	TinyML	TinyML	Edge ML	Cloud ML

The microcontroller column gives the nRF52840 of the Arduino Nano 33 BLE Sense. In the lab you build for the XIAO with PSRAM and with no PSRAM.

Source: Kit pages and chapter 2 of *Machine Learning Systems*; data sheets of the nRF52840 and of the ESP32-S3.

The memory of the four devices



- ▶ From the microcontroller to the XIAO with PSRAM: a factor of 32.
- ▶ From the XIAO to the Raspberry Pi 5: a factor of about 1000.
- ▶ From the microcontroller to the cloud GPU: a factor of about 300 000.

Key point: A model that fits one device does not fit the next smaller device. You design the model for the target device.

Exercise: select a paradigm

For each application, select one paradigm: cloud, edge, mobile, or TinyML. State the constraint that decides.

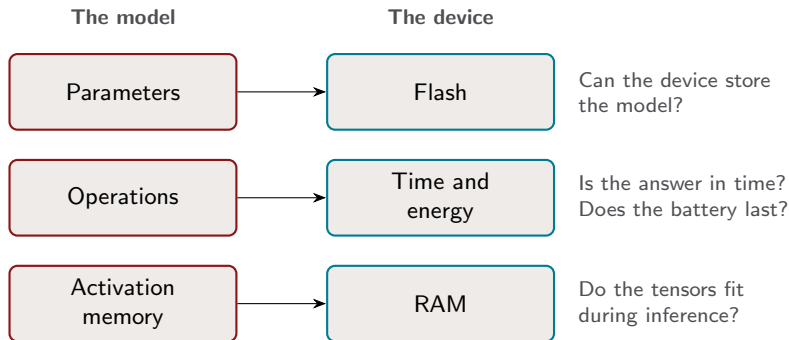
1. **A sensor on a water pipe in the desert.** It listens for the sound of a leak. It has a battery that must last one year. The site has no network connection.
2. **A camera above a production line.** It must reject a defective part in 50 ms. The company does not permit the images to leave the factory. The model has a size of 200 MB.
3. **A search tool for the document archive of a company.** It uses a large language model to answer questions. A user accepts an answer after two seconds.

Work with your neighbour for 5 minutes. Use the four filters: privacy, latency, computation, cost.

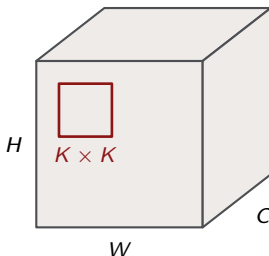
1. **TinyML.** The energy decides: only a microcontroller operates for one year on a battery. Availability also removes the cloud, because the site has no connection.
2. **Edge ML.** Privacy removes the cloud. The latency of 50 ms permits a local computer. The model of 200 MB removes TinyML. The camera has a fixed position and mains power, so a mobile device gives no advantage.
3. **Cloud ML.** The computation decides: a large language model does not fit a small device. A latency of two seconds is acceptable. If the documents must stay in the company, the model runs on a server of the company.

More than one constraint can apply. The filters remove the paradigms that are not possible.

A model has three budgets



Key point: The three budgets are different numbers. A model can meet two budgets and fail the third.



- ▶ A feature map has $H \times W \times C$ values: height, width, channels.
- ▶ A kernel has the size $K \times K$.
- ▶ S is the stride. P is the padding.

Output size of a convolution or a pooling layer

$$H_{\text{out}} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1$$

The same formula gives W_{out} .

Example: $H = 96$, $K = 3$, $S = 2$, $P = 1$.

$$H_{\text{out}} = \lfloor 95/2 \rfloor + 1 = 48$$

A layer with N_{in} inputs and N_{out} outputs:

Parameters	$N_{in} \times N_{out} + N_{out}$
Operations	$N_{in} \times N_{out}$ MACs
Output size	N_{out} values

A MAC is one multiply-accumulate operation: $a \leftarrow a + w \times x$.

Example: 784 inputs, 128 outputs

- ▶ Weights:
 $784 \times 128 = 100\,352$
- ▶ Biases: 128
- ▶ Parameters: 100 480
- ▶ Operations: 100 352 MACs
- ▶ Output: 128 values

Key point: Each weight is used one time. The number of operations is equal to the number of weights. The layer is expensive in flash.

C_{out} filters, each of size $K \times K \times C_{\text{in}}$:

Parameters	$K^2 C_{\text{in}} C_{\text{out}} + C_{\text{out}}$
Operations	$K^2 C_{\text{in}} C_{\text{out}} \times H_{\text{out}} W_{\text{out}}$ MACs
Output size	$H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}}$ values

Example: 3×3 , 32 to 64 channels, output 56×56

- ▶ Weights:
 $9 \times 32 \times 64 = 18\,432$
- ▶ Parameters: 18 496
- ▶ Operations:
 $18\,432 \times 3136 \approx 57.8$ million MACs
- ▶ Output: 200 704 values

Key point: The layer uses each weight again at each output position. It is small in flash, but expensive in operations and in RAM.

The two layer types have opposite costs

	Fully connected	Convolution
The parameters grow with	the number of inputs	the kernel size and the channels
A larger image gives	more parameters	the same parameters, more operations
Operations for each weight	1	$H_{\text{out}} \times W_{\text{out}}$
The main cost	flash	operations and RAM

- ▶ Example: a fully connected layer from 1024 inputs to 1024 outputs has 1 048 576 weights.
- ▶ A convolution layer with a 3×3 kernel shares its weights across the image. Its parameters do not depend on the image size.

Layer	Parameters	Effect on the cost
Activation (ReLU)	0	One comparison for each value. The size does not change.
Pooling, stride 2	0	The output has one quarter of the values. All later layers cost less.
Global average pooling	0	$H \times W \times C$ values become C values. It replaces a large fully connected layer.
Batch normalization	$2C$	After training, the converter merges it into the convolution before it. Its cost at inference is then zero.

Key point: A stride of 2 early in the network is the cheapest method to decrease the operations and the RAM of all later layers.

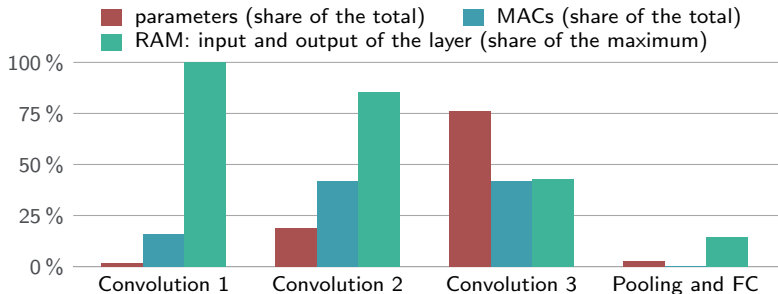
Example: a small network, layer by layer

Input $96 \times 96 \times 3$. Each convolution: 3×3 , stride 2, padding 1.

Layer	Output	Values	Parameters	MACs
Input	$96 \times 96 \times 3$	27 648		
Convolution 1, 16 filters	$48 \times 48 \times 16$	36 864	448	995 328
Convolution 2, 32 filters	$24 \times 24 \times 32$	18 432	4640	2 654 208
Convolution 3, 64 filters	$12 \times 12 \times 64$	9216	18 496	2 654 208
Global average pooling	64	64	0	
Fully connected	10	10	650	640
Total			24 234	6 304 384

Calculate one row with the formulas before you continue.

Example: where the costs are



- ▶ The first layer has 2 % of the parameters, but it needs the most RAM.
- ▶ The last convolution has 76 % of the parameters.

Key point: The early layers set the RAM budget. The late layers set the flash budget.

Rule

Model size $\approx$ number of parameters $\times$ bytes for each parameter.

- ▶ float32: 4 bytes
- ▶ int8: 1 byte (Day 4)
- ▶ The model file also holds the structure of the network. This part is small.

Model	Parameters	float32
Example network	24 234	95 KB
Keyword spotting	26 000	< 100 KB
MobileNetV2	3.5 million	13.4 MB
ResNet-50	25.6 million	98 MB

On a microcontroller, the flash holds the program and the model. The weights stay in the flash. The device does not copy them to the RAM.

Source: Parameters of the three models: *Machine Learning Systems*, Volume I, chapters 2 and 6.
Sizes with 1 MB = 1024 KB.

- ▶ Count the MACs of each layer and add them.
- ▶ One MAC is two floating-point operations (FLOPs).
- ▶ Many papers write “FLOPs” for the number of MACs. Read the definition before you compare.
- ▶ More operations need more time and more energy on the same device.

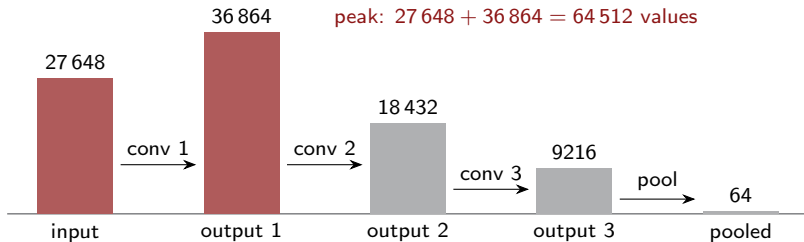
The count is an estimate

- ▶ The time also depends on memory access, on the kernel library, and on the data type.
- ▶ Example of the book: MobileNetV2 has 14 times fewer operations than ResNet-50, but it can be slower on a data-centre GPU.

Key point: Use the count to compare models. Then measure the latency on the device (Day 9).

Source: *Machine Learning Systems*, Volume I, chapter 6.

Budget 3: activation memory and RAM



- ▶ When a layer runs, its input and its output are in the RAM at the same time.
- ▶ After the layer, the runtime uses the memory of the input again.
- ▶ The peak activation memory is the largest sum over all layers.

The same network in two data types

	float32	int8
Bytes for each value	4	1
Model size (24 234 parameters)	95 KB	24 KB
Peak activation memory (64 512 values)	252 KB	63 KB

A microcontroller with 256 KB of RAM:

- ▶ float32: the tensors need 252 KB. No RAM remains for the program, so the model does not fit.
- ▶ int8: the tensors need 63 KB. The model fits.

Key point: This model is small in flash in the two data types. The RAM decides. You cannot see this from the number of parameters.

The budgets are independent

One layer	Parameters	Model size	Output tensor
Fully connected, 1024 to 1024	1 049 600	4.0 MB	4 KB
Convolution 3×3 , 3 to 16 channels, image 224×224	448	2 KB	3.1 MB

All sizes are for float32.

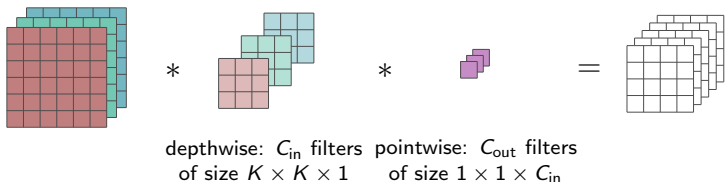
- ▶ The first layer needs much flash and almost no RAM.
- ▶ The second layer needs almost no flash and much RAM.
- ▶ A pruned model has fewer parameters. Its tensors can have the same size as before.

Key point: Report three numbers for each model: parameters, operations, and peak activation memory.

Standard convolution: one step



Depthwise separable convolution: two steps



The depthwise step filters the space. The pointwise step mixes the channels. The output has the same size in the two cases.

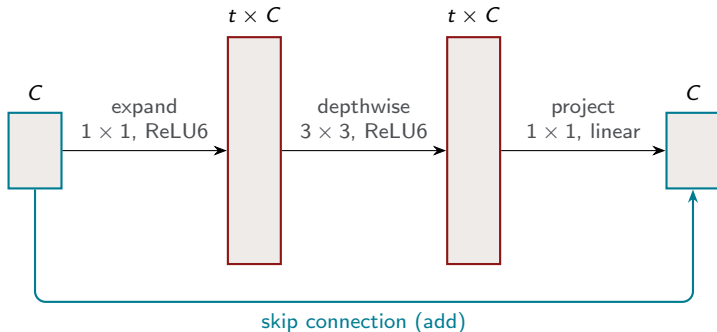
	Weights	3×3 , 32 to 64 channels
Standard convolution	$K^2 C_{\text{in}} C_{\text{out}}$	18 432
Depthwise step	$K^2 C_{\text{in}}$	288
Pointwise step	$C_{\text{in}} C_{\text{out}}$	2048
Separable, total	$K^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}$	2336

$$\frac{\text{separable}}{\text{standard}} = \frac{1}{C_{\text{out}}} + \frac{1}{K^2} = \frac{1}{64} + \frac{1}{9} \approx \frac{1}{7.9}$$

- ▶ The operations decrease by the same factor, because the output size is the same.
- ▶ The output tensor is the same. The RAM does not decrease.

Source: The book gives a factor of 8 to 9 for 3×3 kernels: *Machine Learning Systems*, Volume I, chapter 6.

Efficient block 2: the inverted residual



- ▶ The block of MobileNetV2. The expansion factor t is usually 6.
- ▶ The 3×3 filter operates on many channels, but it is a depthwise filter, so it is cheap.
- ▶ The skip connection joins the two narrow tensors. A classic residual block joins the wide tensors. Thus the name “inverted”.

Source: Sandler et al., “MobileNetV2”, 2018.

Inverted residual: cheap in flash, expensive in RAM

Example: $C = 32$, $t = 6$, feature map 28×28 , stride 1.

Weights

- ▶ Expand: $32 \times 192 = 6144$
- ▶ Depthwise: $9 \times 192 = 1728$
- ▶ Project: $192 \times 32 = 6144$
- ▶ Total: 14 016

A standard 3×3 convolution on 192 channels: 331 776 weights, 24 times more.

Tensors

- ▶ Input: $28 \times 28 \times 32 = 25\,088$ values
- ▶ Expanded: 150 528 values
- ▶ During the depthwise step: two expanded tensors and the input for the skip connection
- ▶ Peak: 326 144 values. This is 13 times the input.

Key point: The block saves parameters and operations. It does not save RAM. On a microcontroller, the expanded tensors are often the limit.

Source: Calculated for this course, with separate buffers for the input and the output of each layer.

The result: two models for the same task

	ResNet-50	MobileNetV2
Parameters	25.6 M	3.5 M
Operations	4.1 G	300 M
Size, float32	98 MB	13.4 MB

Both models classify ImageNet images.

- ▶ 7 times fewer parameters.
- ▶ 14 times fewer operations.
- ▶ The cause is the block design, not a smaller input image.
- ▶ MobileNetV2 still does not fit a microcontroller with 256 KB of RAM. Block 3 shows why.

Key point: The design of the blocks decides the cost before you apply quantization or pruning.

Source: Parameters and operations: *Machine Learning Systems*, Volume I, chapter 6. Sizes with 1 MB = 1024 KB.

1. Write the output size of each layer: $H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}}$.
2. Calculate the parameters of each layer. Add them.
→ model size → compare with the flash.
3. Calculate the MACs of each layer. Add them.
→ first estimate of the latency and of the energy.
4. For each layer, add the size of its input and of its output. Take the maximum.
→ peak activation memory → compare with the RAM.

In the lab

A notebook does these four steps for three models. You compare the results with the budgets of four devices.

Calculate the output size and the number of parameters of each layer. Include the biases.

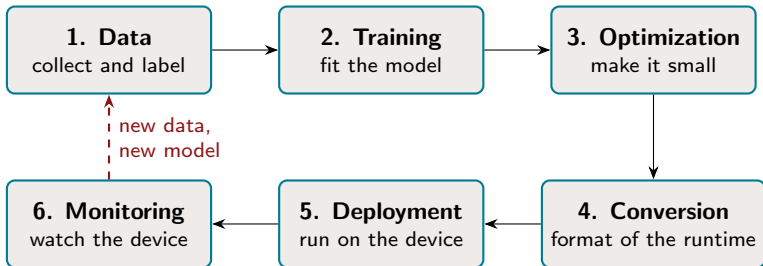
1. **Convolution.** Input $96 \times 96 \times 3$. 16 filters of size 3×3 , stride 2, padding 1.
2. **Depthwise separable convolution.** Input $48 \times 48 \times 16$. Depthwise step: 3×3 , stride 1, padding 1. Pointwise step: 32 filters.
3. **Fully connected layer.** Input: 32 values from a global average pooling layer. Output: 10 classes.

Then answer: how many parameters does a standard 3×3 convolution need in place of layer 2?

Work alone for 7 minutes. Use no calculator.

1. Output: $\lfloor (96 + 2 - 3)/2 \rfloor + 1 = 48$, so $48 \times 48 \times 16$.
Parameters: $3 \times 3 \times 3 \times 16 + 16 = 448$.
2. Output: $48 \times 48 \times 32$. The stride is 1 and the padding keeps the size.
Depthwise: $3 \times 3 \times 16 + 16 = 160$. Pointwise:
 $16 \times 32 + 32 = 544$.
Parameters: $160 + 544 = 704$.
3. Output: 10 values. Parameters: $32 \times 10 + 10 = 330$.

Standard convolution in place of layer 2: $3 \times 3 \times 16 \times 32 + 32 = 4640$ parameters. The separable layer needs 6.6 times fewer.



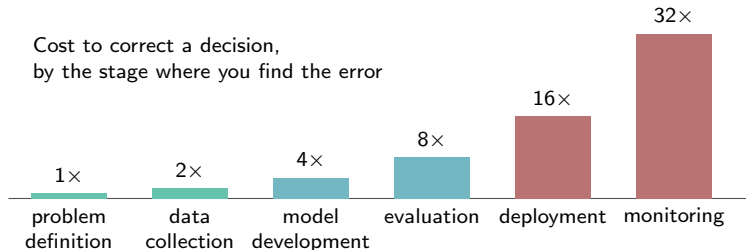
- ▶ A cloud project trains a model and serves it. An edge project adds two stages: optimization and conversion.
- ▶ The workflow is a loop. Deployment is the start of the feedback, not the end of the project.

Each stage has a place and an output

Stage	Where it runs	Output
1. Data	Sensor of the device	A labelled data set
2. Training	Laptop or cloud	A float32 model with a known accuracy
3. Optimization	Laptop or cloud	A smaller model. You measure its accuracy again.
4. Conversion	Laptop	A model file for the runtime of the device
5. Deployment	Device	An application: sensor input, model, action
6. Monitoring	Device and network	Measured values, logs, new data

Key point: Training runs where the computation is. Inference runs where the data is. This pattern is the train-serve split.

The device constraints come first



- ▶ The book gives a rule: the cost of a correction doubles with each stage.
- ▶ Example: after training, you find that the model needs 300 KB of RAM. The device has 256 KB. You start the model design again.

Key point: Write the budgets of the device before you collect data: flash, RAM, latency, energy.

Source: *Machine Learning Systems*, Volume I, chapter 3.

- ▶ The data set is the critical part of the workflow.
- ▶ Record the data with the same type of sensor that the device uses.
- ▶ Record in the conditions of the application: position, noise, light, different persons.
- ▶ Add classes for “nothing” and for “unknown”. The device sees these inputs most of the time.

Questions of the device

- ▶ Which sampling rate can the sensor give?
- ▶ How long is one input window?
- ▶ How large is one input in RAM?

In this course

Day 2: you record motion data with the IMU of the XIAOMI Kit.

- ▶ Training runs on a laptop or in the cloud. It needs the backward pass and many passes through the data.
- ▶ The device does only the forward pass.
- ▶ Select a model that can meet the three budgets of the device (block 2). A model that is too large stays too large.
- ▶ Keep the pre-processing identical in training and on the device.

Output of this stage

- ▶ A float32 model
- ▶ The accuracy on test data
- ▶ Parameters, operations, and peak activation memory

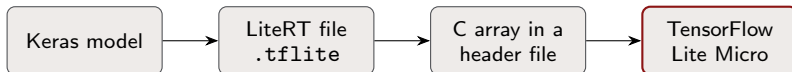
In this course

Days 3, 5, and 8: you train small models with Keras, Edge Impulse Studio, and Ultralytics.

Method	What it changes	In this course
Quantization	The data type: <code>float32</code> to <code>int8</code> . Four times less flash and RAM.	Day 4
Pruning	It removes weights or channels.	Day 6
Knowledge distillation	A small model learns from a large model.	Day 6
Efficient design	Blocks with a lower cost (block 2).	Days 1 and 6

- ▶ Each method exchanges accuracy for size, latency, or energy.
- ▶ Measure the accuracy and the three budgets before and after each step.

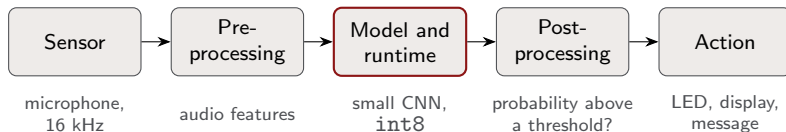
Microcontroller (XIAOML Kit)



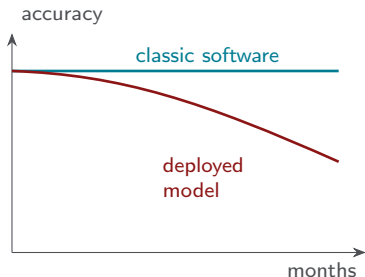
Linux device (Raspberry Pi 5)



- ▶ The training framework does not run on a small device. A runtime reads the converted model and executes it.
- ▶ A microcontroller has no file system for the model. The model is a constant array in the firmware.
- ▶ Risk: the runtime does not support an operator of the model. Check this before you train for a long time.



- ▶ The device runs a pipeline, not only a model. Each step needs time and memory.
- ▶ Microcontroller: one firmware image holds the program, the runtime, and the model.
- ▶ Linux device: a program loads the runtime and the model file.
- ▶ Then you measure on the device: latency, memory, and energy (Day 9).

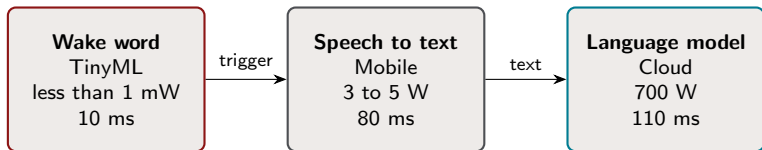


- ▶ Classic software fails with an error message.
- ▶ A model fails with no message. The world changes, and the accuracy decreases.
- ▶ The book gives a loss of 10 to 15 % in some months, with no change of the code.
- ▶ An edge device has no operator who watches it. It must report its condition.

Key point: Send measured values from the device: confidence, latency, temperature. Days 12 and 13 show the method.

Source: *Machine Learning Systems*, Volume I, chapter 1. The curve is a sketch, not measured data.

Hybrid architectures: one product, three paradigms



- ▶ A voice assistant is three systems. Each stage runs where its constraint costs the least.
- ▶ The wake word model is always on. Thus it must use very little power.
- ▶ The language model is too large for the device. Thus it runs in the cloud.
- ▶ The total budget is 200 ms. The network is the part that varies most.

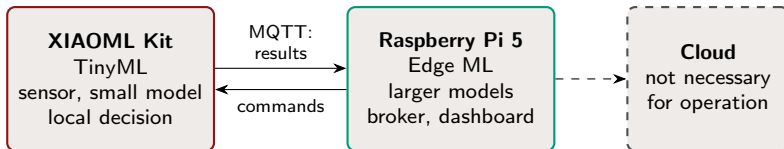
Source: *Machine Learning Systems*, Volume I, chapters 1 and 2.

Three patterns for a hybrid system

Pattern	What it does	Select it when
Train-serve split	Training in the cloud, inference on the device.	Training needs a scale that inference does not need.
Hierarchical processing	Each tier filters the data and sends less to the next tier.	The data volume is larger than the capacity of the connection.
Progressive deployment	One model family, compressed for each tier.	The same function is necessary on devices of different size.

A special case of hierarchical processing: a small model on the device answers when its confidence is high. It sends only the difficult inputs to a larger model.

The hybrid system of this course



- ▶ Week 1: a model on the microcontroller makes a decision at the sensor.
- ▶ Week 2: the Raspberry Pi runs models that are too large for the microcontroller.
- ▶ Week 3: the two boards exchange messages. The system continues to work with no internet connection.
- ▶ You train on a laptop and deploy on the boards. This is the train-serve split.

Week 1: XIAOML Kit

Day 1
Landscape
and budgets

Day 2
Data

Day 3
Train, convert,
deploy

Day 4
Optimize:
quantization

Day 5
Deploy:
audio, vision

Week 2: laptop and Raspberry Pi 5

Day 6
Optimize: prune,
distil

Day 7
Convert:
runtimes

Day 8
Train, deploy:
detection

Day 9
Measure:
benchmarks

Day 10
Deploy:
language models

Week 3: both boards

Day 11
Connect:
RTSP

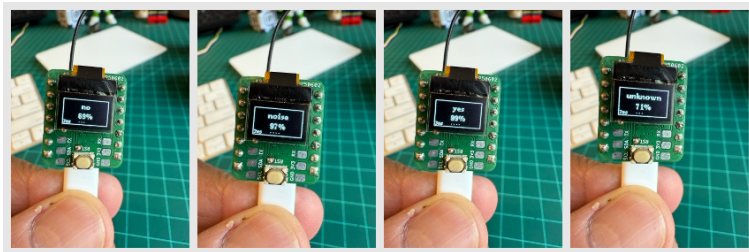
Day 12
Connect:
MQTT

Day 13
Monitor

Day 14
Design for
production

Day 15
Capstone

Key point: Each day adds one stage of the workflow, or it measures a stage. The capstone project uses all six stages.



The kit listens continuously. It classifies one second of sound: YES, NO, NOISE, or UNKNOWN. The display shows the class and the confidence.

Predict before the demonstration

1. Does it work with Wi-Fi off?
2. How much time passes from the word to the result?
3. What does it show for a different word?

Source: Photograph: kit chapter "Keyword Spotting" of *Machine Learning Systems* (CC BY-NC-SA 4.0).

The demonstration through the six stages

Stage	Keyword spotting on the kit
1. Data	Speech Commands data set and own recordings. Audio with 16 kHz and 16 bits. Four classes.
2. Training	Edge Impulse Studio. Audio features (MFCC) and a small CNN with two convolution blocks.
3. Optimization	Quantization to int8.
4. Conversion	The Studio makes an Arduino library with the model and TensorFlow Lite Micro.
5. Deployment	One sketch: microphone, features, model, threshold, display. The build needs PSRAM.
6. Monitoring	The display shows the class and the confidence. No data leaves the kit.

The source reports an accuracy of about 87 % on its test data. You build this system on Day 5.

Source: Kit chapter “Keyword Spotting” of *Machine Learning Systems*.

Before you deploy: does the model fit?

Flash

model size + program $\leq$ flash
model size = parameters $\times$ bytes
for each parameter

RAM

peak activation memory +
program data $\leq$ RAM
peak = values $\times$ bytes for each
value

- ▶ The program also needs RAM: the stack, the sensor buffer, the runtime.
- ▶ Do not use the complete RAM for the tensors. In the lab you measure the RAM that a sketch with no model uses.
- ▶ Check the two budgets. One number is not sufficient.

Key point: Device of the exercise: a Cortex-M4 microcontroller with 1 MB of flash and 256 KB of RAM.

Exercise: which model fits in 256 KB of RAM?

The device has 256 KB of RAM and 1 MB of flash.

Model	Type	Parameters	Peak (values)
A. Fully connected, 784-128-64-10	float32	109 386	912
B. Example CNN of block 2	float32	24 234	64 512
C. MobileNetV2, image 224×224	int8	3.5 million	1.5 million

1. Predict first, with no calculation: which models fit in the RAM?
2. Calculate the peak activation memory and the model size of each model in KB.
3. Which model has the most parameters of the models that fit?

Work with your neighbour for 7 minutes.

Answer: which model fits in 256 KB of RAM?

Model	Peak in RAM	Model size	Result
A	3.6 KB	427 KB	fits
B	252 KB	95 KB	RAM too small
C	1470 KB	3.3 MB	RAM and flash too small

- ▶ A has 4.5 times more parameters than B, but only A fits. The parameters do not show the RAM.
- ▶ B needs 252 KB for the tensors. No RAM remains for the program. In `int8`, B needs 63 KB and it fits.
- ▶ C fails in the first layers. Its largest tensors have 1.5 million values. Quantization is not sufficient: the model needs a smaller image or a different design.

- ▶ Latency, bandwidth, privacy, cost, and availability move inference to the edge. The four paradigms are cloud, edge, mobile, and TinyML.
- ▶ A model has three budgets: parameters, operations, and activation memory. A model can meet one budget and fail a different one.
- ▶ Depthwise separable convolutions and inverted residuals decrease the parameters and the operations. They do not decrease the RAM.
- ▶ The edge AI workflow has six stages: data, training, optimization, conversion, deployment, and monitoring.

Key point: Compare the flash and the RAM of the device with the size and the peak activation memory of the model before you deploy.

Book chapters

- ▶ *Machine Learning Systems* (mlsysbook.ai), Volume I: chapter 1 “Introduction” and chapter 2 “ML Systems”.
- ▶ *Machine Learning Systems*, hardware kits: “Hardware Platforms” and the “Setup” chapter of the XIAOML Kit.
- ▶ *XIAO: Big Power, Small Board*, chapter 4.1 “Understanding TinyML and Edge Impulse Studio”.

Papers

- ▶ “MobileNetV2: Inverted Residuals and Linear Bottlenecks”, Sandler et al., 2018.
- ▶ “MCUNet: Tiny Deep Learning on IoT Devices”, Lin et al., 2020.

Tools

- ▶ Arduino IDE 2: <https://docs.arduino.cc/software/ide/>
- ▶ Arduino core for the ESP32:
<https://docs.espressif.com/projects/arduino-esp32/>

This lecture adapts material from these sources:

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.
- ▶ *TinyML Made Easy: XIAO ESP32S3* by Marcelo Rovai. github.com/Mjrovai/XIAO-ESP32S3-Sense, Apache-2.0.
- ▶ *XIAO: Big Power, Small Board* by Lei Feng and Marcelo Rovai. github.com/Mjrovai/XIAO_Big_Power_Small_Board-ebook, GPL-3.0.

Material from a source with the licence CC BY-NC-SA 4.0 keeps that licence.